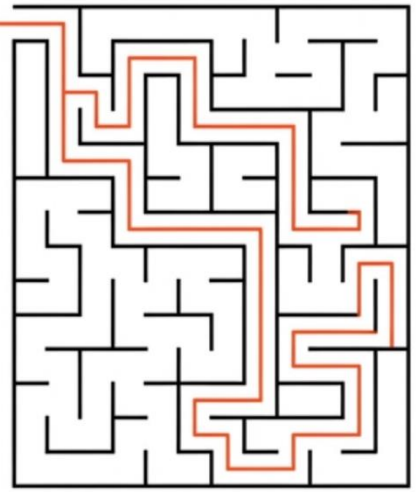
Universidad del Rosario | Escuela de Ciencias e Ingeniería | MACC

```
void solveMaze(x, y) {
  if (isGoal(x, y)) return;
  ...
  solveMaze(nextX, nextY);
}
```

Recursión: Backtracking



Objetivos

- **Comprender la Pila de Llamadas (Call Stack):** Visualizar cómo el sistema operativo asigna memoria (Stack Frames) para cada llamada recursiva y entender el riesgo de *Stack Overflow*.
- **Dominar el Estado en C++:** Diferenciar el impacto en memoria y rendimiento al pasar estructuras de datos (como **std::vector**) por valor frente a pasarlas por referencia en funciones recursivas.
- **Transicionar a la Recursión de Ramificación:** Pasar de la recursión lineal (un solo camino) a explorar un árbol de decisiones (múltiples caminos), sentando las bases directas para el *backtracking*.

Inducción Matemática

Caso Base:
Demostrar para $n = 1$.

Paso Inductivo:
Asumir n, probar $n + 1$.

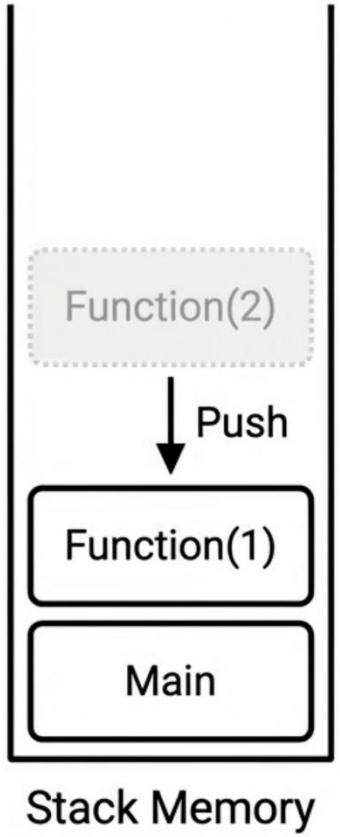
Recursión en Código

```
if (n == 0) return;
// La condición de parada
```

```
func(n - 1);
// El problema se reduce
```

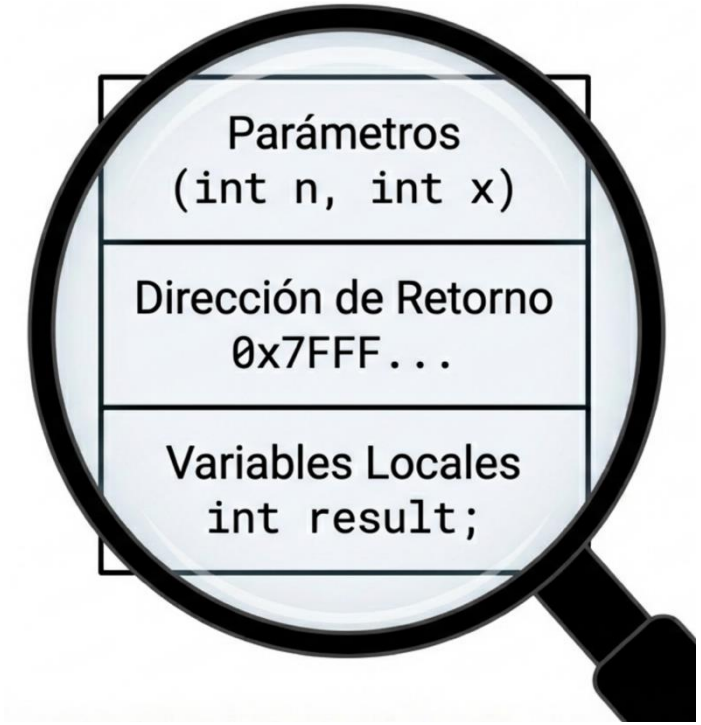
La recursión es inducción matemática ejecutada por el procesador.

El motor oculto: Stack Frames



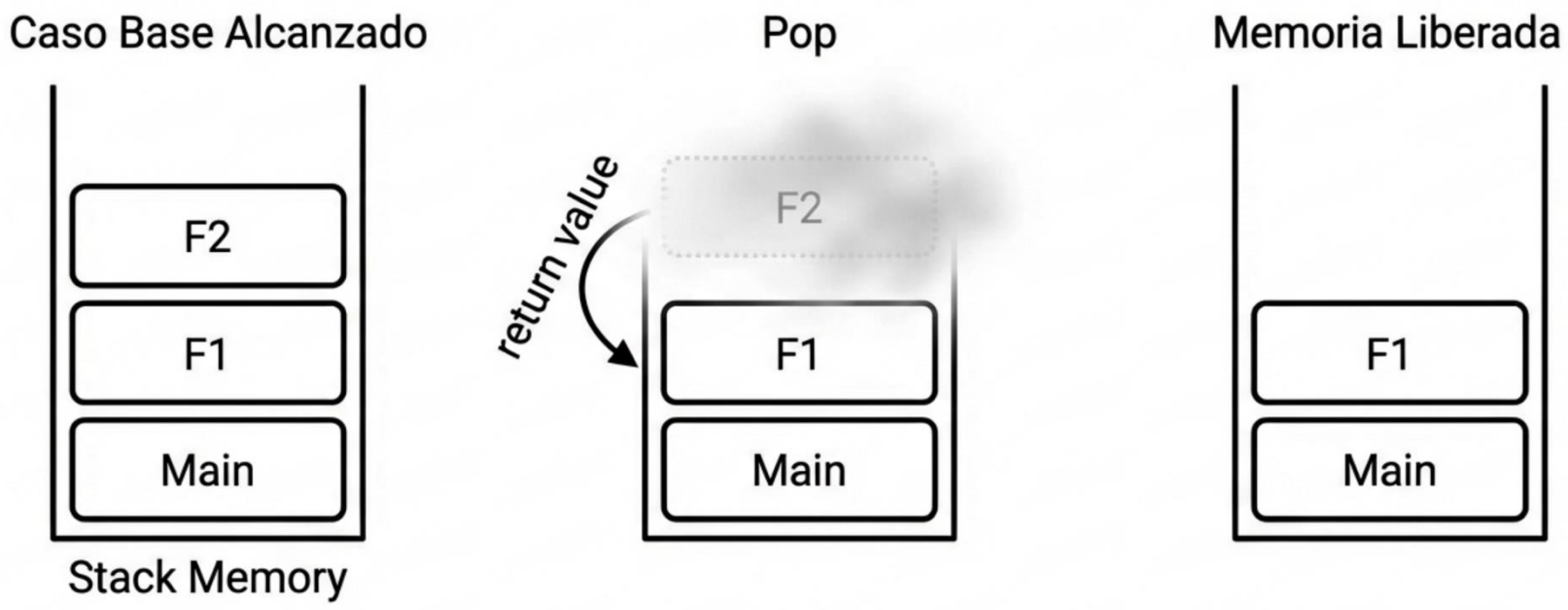
- LIFO (Last-In, First-Out): El último elemento en entrar es el primero en salir.
- Cada llamada a una función reserva un bloque nuevo de memoria estática.
- La memoria crece con cada nivel de recursión.

Stack Frame



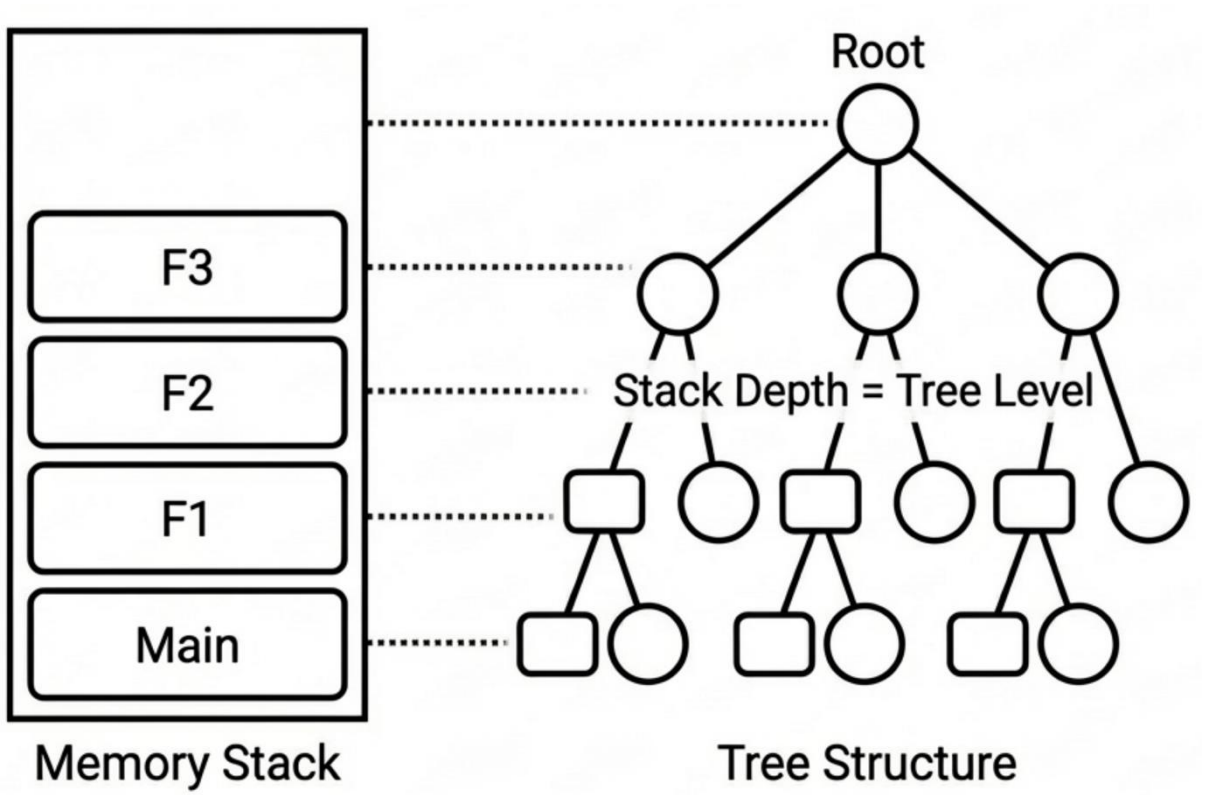
Cada llamada recursiva posee su propio universo de variables. No se mezclan con las llamadas anteriores.

El Retorno: Stack Unwinding



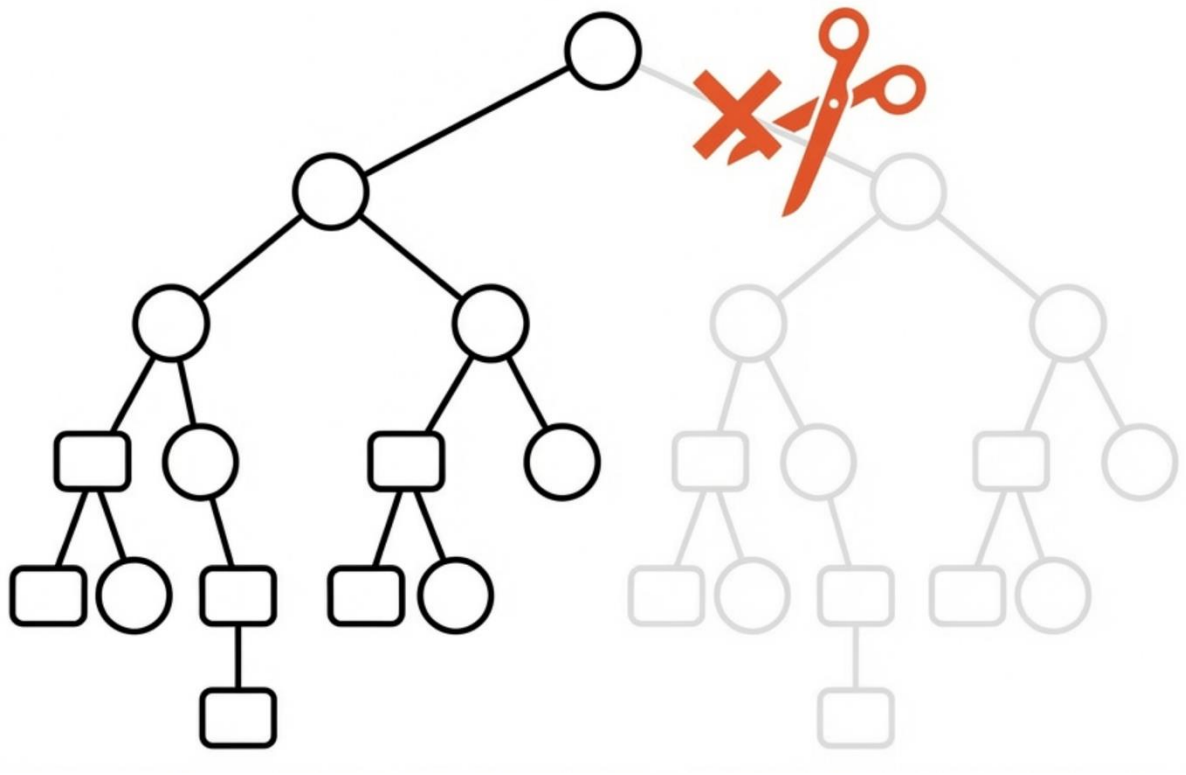
Al tocar el Caso Base, la expansión se detiene. El valor calculado se pasa al frame inferior como si la función acabara de terminar.

Backtracking



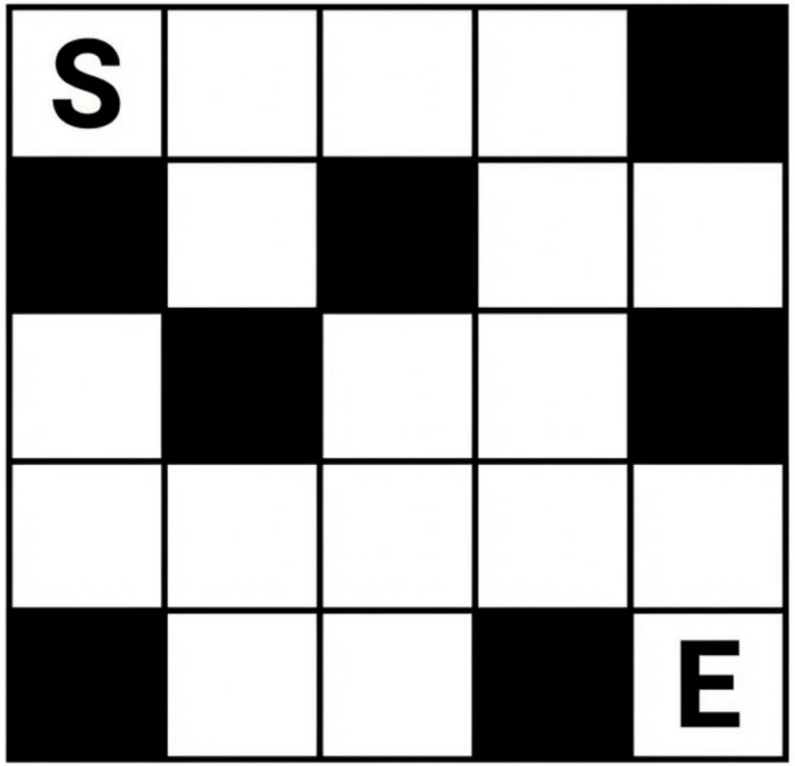
- La recursión es el mecanismo; el Backtracking es la estrategia.
- Visualizamos el problema como un Árbol de Decisiones.
- DFS (Depth-First Search): Exploramos una rama completa hasta el fondo antes de retroceder.

La Poda (Pruning)



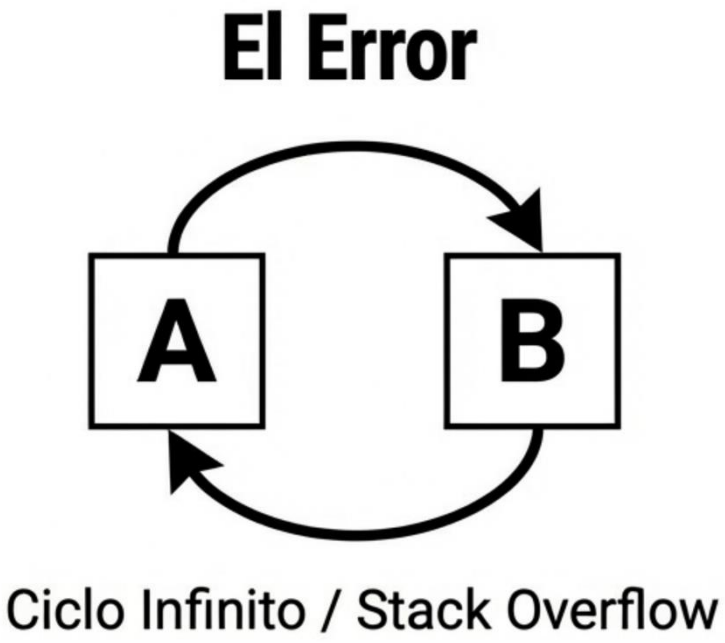
- No todas las ramas merecen ser exploradas.
- Criterio de Poda: Si el estado actual es inválido (ej. una pared), descartamos toda la descendencia de ese nodo.
- Ahorro masivo de cómputo al eliminar caminos muertos prematuramente.

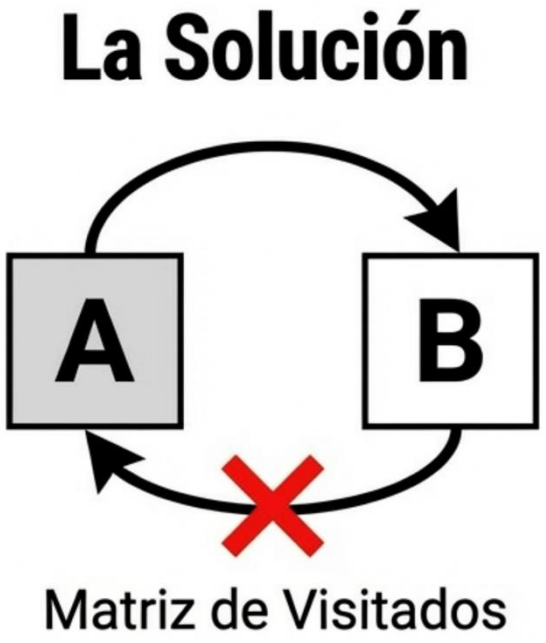
El laberinto



- El laberinto es una matriz M[filas][col].
- Movimientos Válidos: Arriba, Abajo, Izquierda, Derecha.
- Objetivo: Encontrar una secuencia de pasos desde S hasta E.

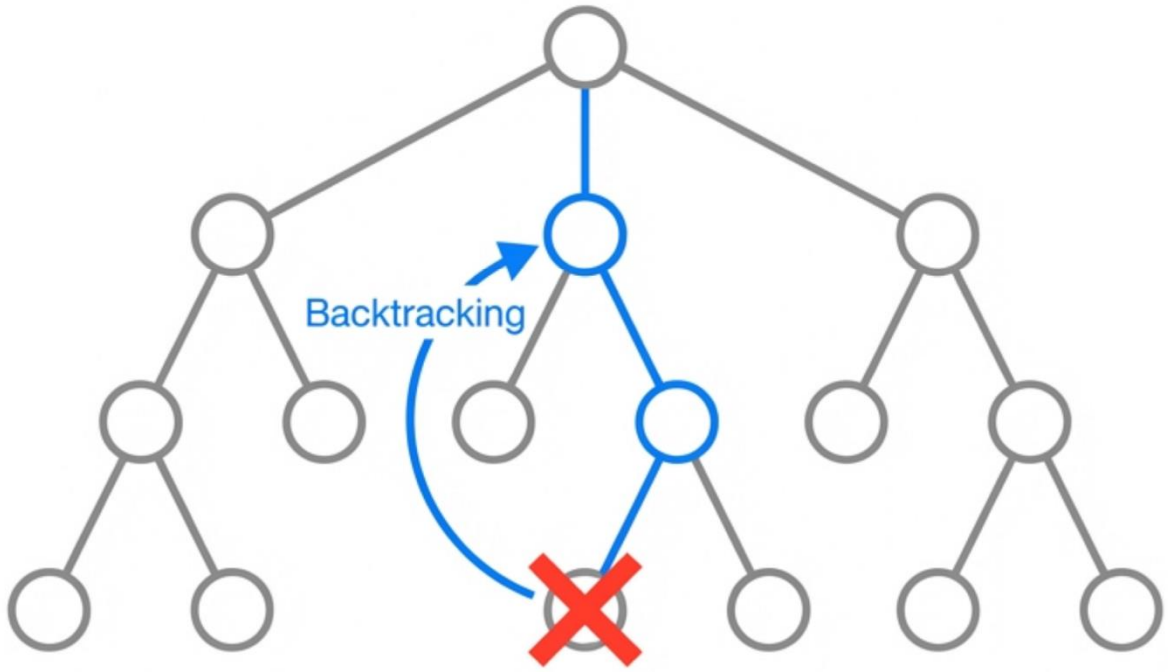
Gestión de Estados: Memoria del Camino





Esencial para convertir un grafo con ciclos en un árbol finito.

Backtracking



Búsqueda sistemática en un árbol de decisión.

El principio del 'Hilo de Ariadna'. Probar, fallar, retroceder y reintentar.

La Recursión

Condiciones de Poda

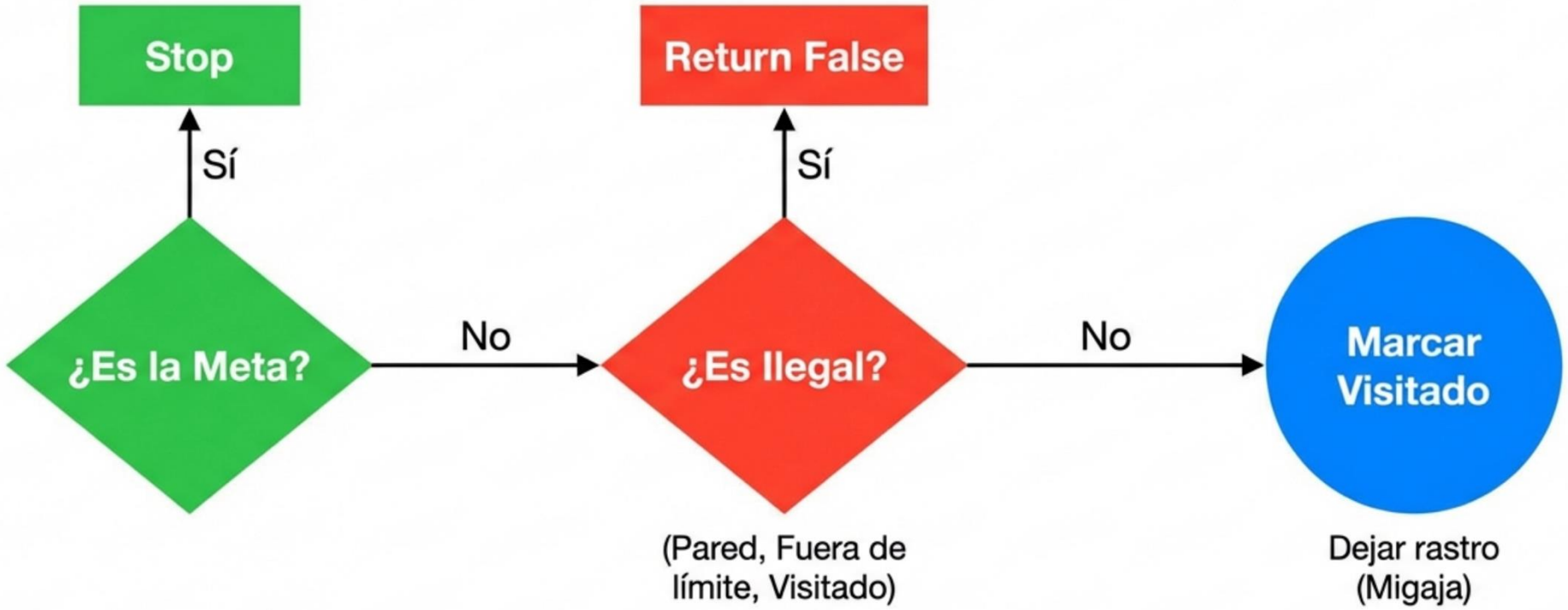
1. ¿Fuera de los límites? → Return False
2. ¿Es una pared? → Return False
3. ¿Ya fue visitado? → Return False

La eficiencia radica en NO explorar lo inútil.

Algoritmo

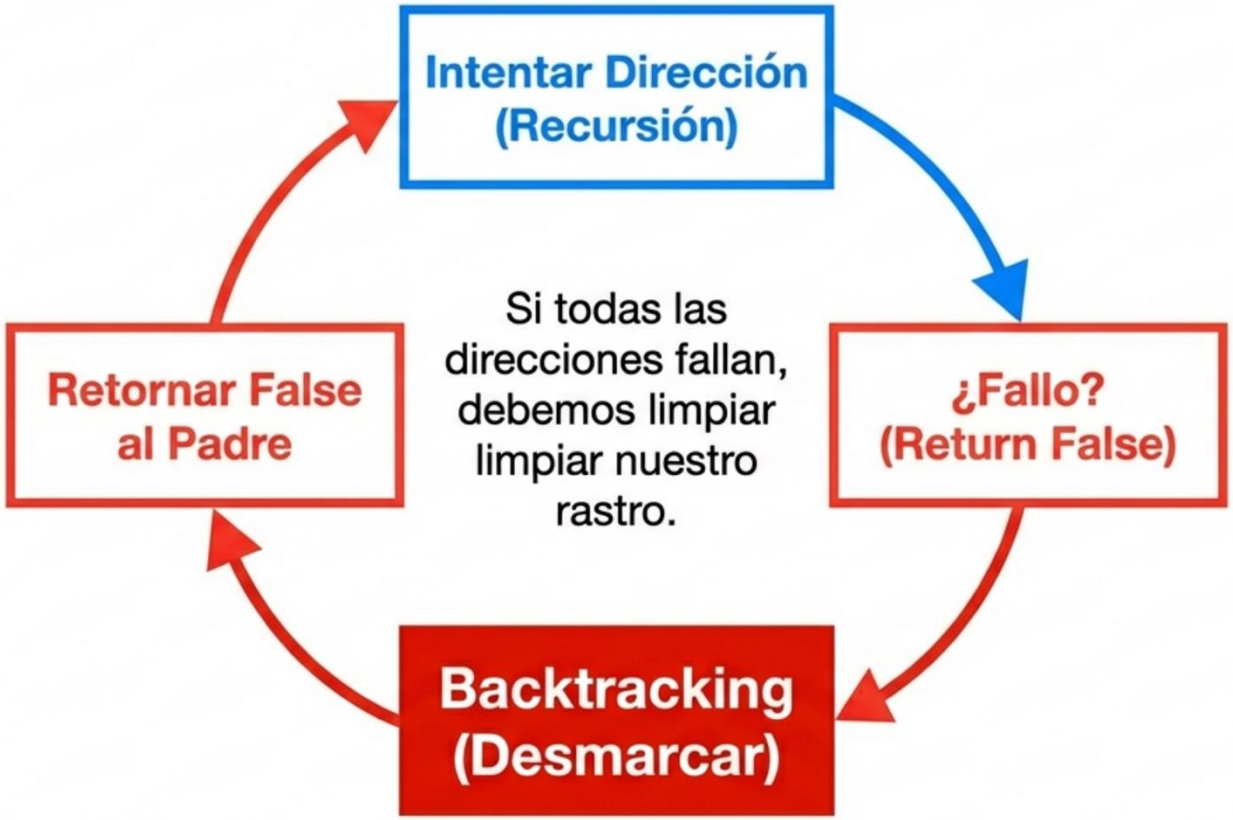
```
ALGORITMO ResolverLaberinto(x, y)

    // 1. EVALUACIÓN DE ÉXITO (Caso Base de Parada)
    SI (x, y) es igual a la Posición_Meta:
        MARCAR (x, y) como parte de la RUTA_SOLUCIÓN
        RETORNAR Verdadero

    // 2. FILTRO DE INVIABILIDAD (Poda y Validación)
    SI (x, y) está fuera de los límites de la matriz O
       (x, y) es un Obstáculo/Pared O
       (x, y) ya ha sido visitado en la rama actual:
        RETORNAR Falso

    // 3. MARCADO DE ESTADO (Exploración)
    MARCAR (x, y) como VISITADO

    // 4. EXPANSIÓN DE RAMAS (Recursión en 4 Direcciones)
    SI ResolverLaberinto(x + 1, y) == Verdadero: RETORNAR Verdadero
    SI ResolverLaberinto(x, y + 1) == Verdadero: RETORNAR Verdadero
    SI ResolverLaberinto(x - 1, y) == Verdadero: RETORNAR Verdadero
    SI ResolverLaberinto(x, y - 1) == Verdadero: RETORNAR Verdadero

    // 5. PASO DE BACKTRACKING (El "Arrepentimiento")
    DESMARCAR (x, y) como VISITADO (Restaurar el estado original)
    RETORNAR Falso

FIN ALGORITMO.
```

Algoritmo

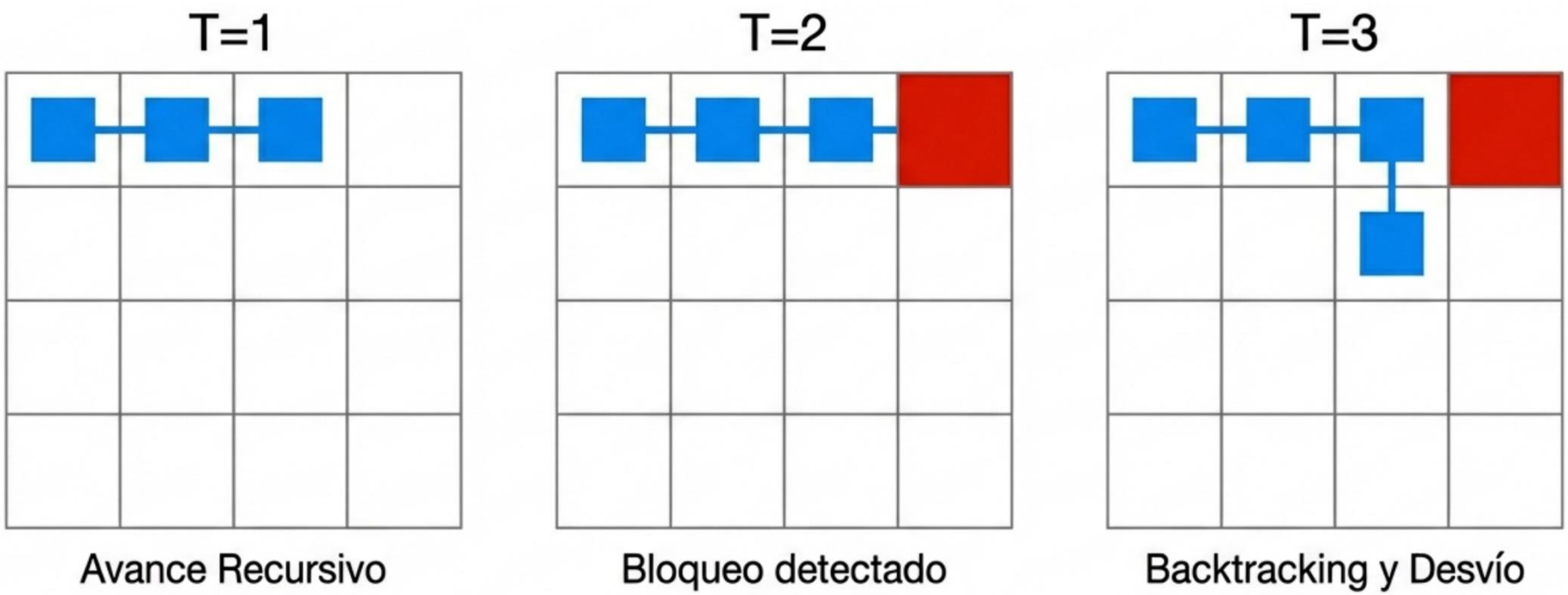


El orden importa. Primero verificamos el éxito, luego el fallo, y solo entonces avanzamos.

Algoritmo


Intentar Dirección (Recursión)
¿Fallo? (Return False)
Backtracking (Desmarcar)
Retornar False al Padre
Si todas las direcciones fallan, debemos limpiar limpiar nuestro rastro.

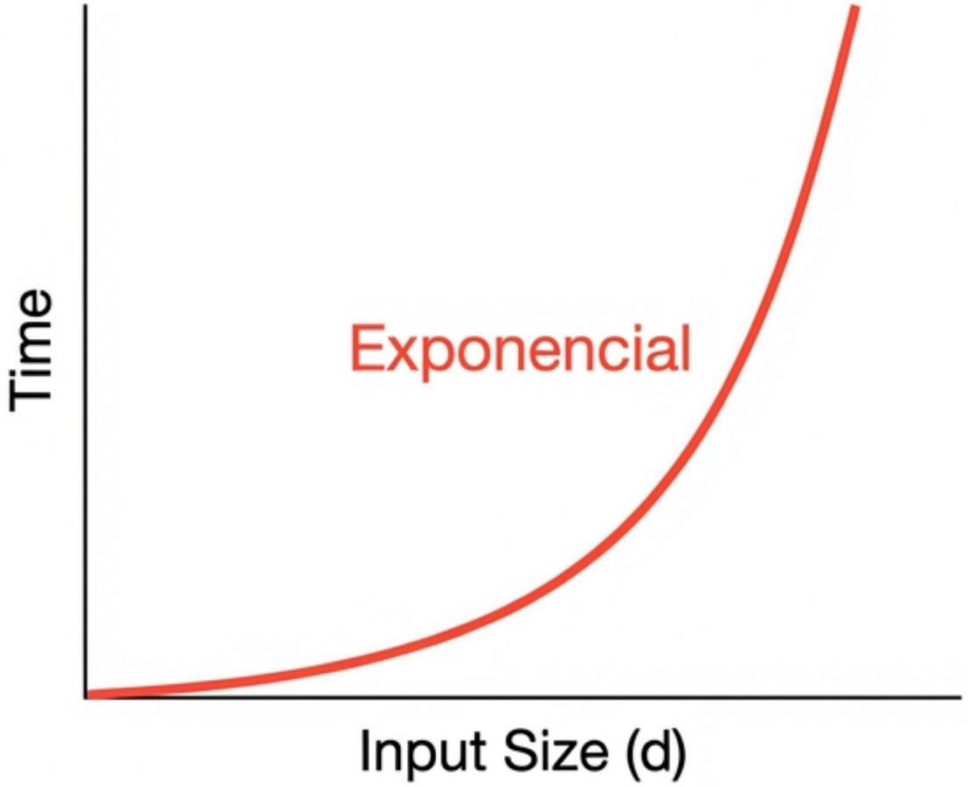
Algoritmo



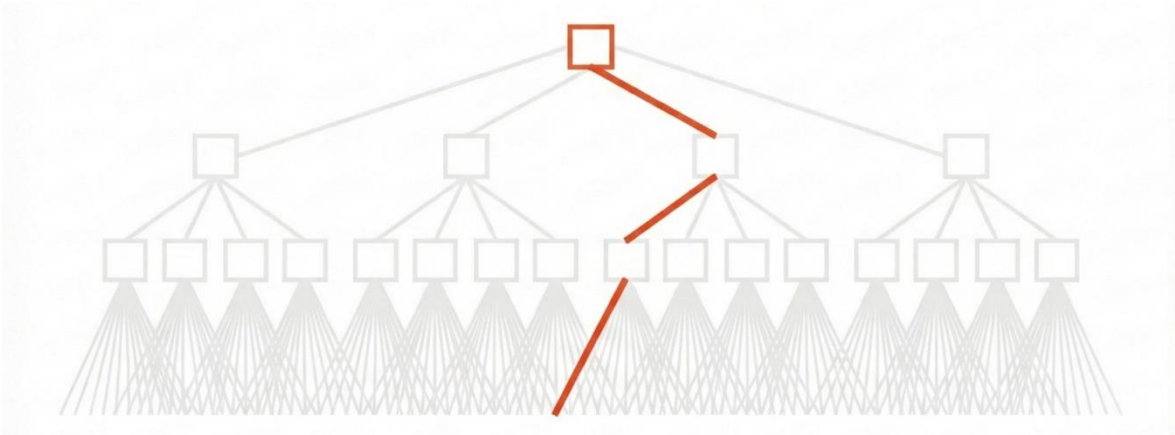
El algoritmo avanza ciegamente hasta chocar, luego retrocede un paso y prueba la siguiente opción.

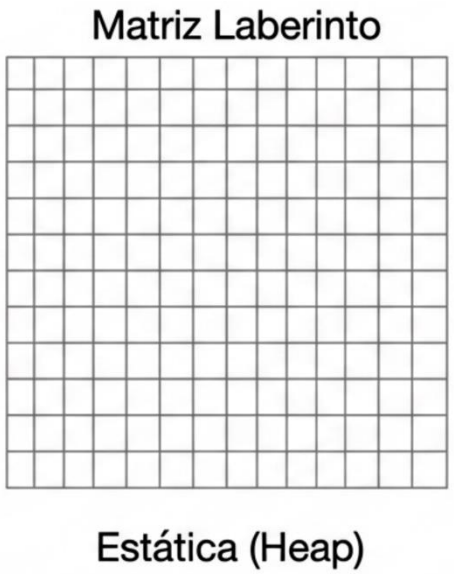
Complejidad Temporal

El tiempo de ejecución del Backtracking puro suele ser **exponencial**. En nuestro laberinto, desde cada celda podemos movernos en 3 direcciones posibles (descontando de la que venimos). En el peor de los casos, un laberinto sin paredes obliga al algoritmo a explorar casi todas las permutaciones de caminos posibles antes de rendirse. Por lo tanto, la complejidad temporal se acota en $O(3^k)$, donde k es el número de celdas libres ($N \times M$).



Complejidad Espacial





Crece linealmente con la profundidad del camino (d).

Aunque exploremos millones de nodos, solo guardamos el camino actual en la pila. DFS es eficiente en memoria.

Resumen

- Vimos que la recursión no es un truco de sintaxis, sino la implementación computacional de la **Inducción Matemática**. Si validamos nuestro caso base y demostramos que el paso inductivo acerca el estado hacia él, el algoritmo es formalmente correcto.

- Entendimos que el Backtracking permite a la computadora "adivinar" un camino y, ante un error, retroceder. La instrucción clave es el **"Desmarcado" de estado** (**lab[x][y] = 0**), lo que restaura el universo para permitir explorar ramificaciones alternativas.

- Terminamos demostrando que todo algoritmo recursivo está inherentemente atado a los límites del *Stack Segment* del sistema operativo, requiriendo un análisis estricto de Complejidad Espacial $O(d)$.